

Dead Reckoning in the Game of Life

On the Computational Hardness of Inverting
Conway’s Cellular Automaton

Justin Fritz

The Canonical Art Corporation

Fort Collins, Colorado

fortcollins.nerdnite.com

June 2026

Abstract

Conway’s Game of Life (CGOL) is a two-dimensional cellular automaton governed by three deterministic rules. Its forward evolution—computing the state of the grid after t steps from a known initial configuration—is trivially efficient, requiring $O(t \cdot n)$ time for an n -cell grid. The inverse problem is not: given a target configuration C_t and a step count t , find an initial configuration C_0 such that $\text{CGOL}^t(C_0) = C_t$. We analyse the computational hardness of this inverse problem as t grows from 10^2 to 10^6 , characterise the information loss at each forward step, and show that the problem transitions from NP-hard-but-feasible to practically undecidable well before $t = 10^6$. The result has implications for the nature of computational irreversibility, the arrow of time in deterministic systems, and the prospects for using CGOL as a cryptographic one-way function.

Contents

1	Introduction	2
2	Background	2
2.1	The Rules	2
2.2	Turing Completeness	2
2.3	Irreversibility	2
3	The Forward Problem	3
4	The Inverse Problem	3
4.1	The $t = 1$ Case	3
4.2	Encoding as SAT	3

5	Hardness by Scale	4
5.1	$t = 10^2$	4
5.2	$t = 10^3$	4
5.3	$t = 10^6$	5
6	The Information-Theoretic View	5
6.1	Entropy Loss at Each Step	5
6.2	The Attractor Landscape	5
7	Connection to Cryptography	6
8	The Arrow of Time in Deterministic Systems	6
9	Summary	7

1. Introduction

Navigation before GPS relied on dead reckoning: given your last known position, your heading, your speed, and elapsed time, estimate where you are now. The mathematics runs forward cleanly. The inverse—given where you are now, determine your exact starting position from an unknown history—is ill-posed. Information was lost in transit.

Conway’s Game of Life presents the same asymmetry in a more extreme form. The forward evolution of any configuration is deterministic and efficiently computable. The backward problem—inferring a specific state t generations into the past—is not merely difficult. For large t , it is, in any practical sense, impossible.

This paper characterises exactly how impossible, and why. We proceed through three scales of t : 10^2 , 10^3 , and 10^6 , showing that the problem transitions through qualitatively distinct regimes of hardness. We connect the result to complexity theory, information theory, and the structure of one-way functions in cryptography.

2. Background

2.1. The Rules

CGOL operates on an infinite (or toroidal) two-dimensional grid of cells, each alive (1) or dead (0). At each generation, every cell is updated simultaneously according to its eight neighbours (Moore neighbourhood):

- **Birth:** a dead cell with exactly 3 live neighbours becomes alive.
- **Survival:** a live cell with 2 or 3 live neighbours survives.
- **Death:** all other live cells die; all other dead cells remain dead.

Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ denote the CGOL transition function on an n -cell grid. The state at generation t is $C_t = f^t(C_0)$, where f^t denotes t -fold composition.

2.2. Turing Completeness

CGOL is Turing complete [1, 4]. This was established by the construction of logic gates from glider collisions and later confirmed by explicit universal Turing machine constructions. Turing completeness has a direct consequence for the inverse problem: any question about the long-run behaviour of a CGOL configuration that requires unbounded simulation is, in general, undecidable. We cannot determine in finite time whether an arbitrary configuration will ever reach a target state.

2.3. Irreversibility

CGOL is not reversible. The mapping f is many-to-one: multiple distinct configurations can evolve to the same successor. Some configurations have no predecessor at all.

Definition 1 (Garden of Eden). *A configuration C is a Garden of Eden if there is no C' such that $f(C') = C$.*

Gardens of Eden were proved to exist by Moore [3] and first explicitly constructed by Banks et al. [2]. Irreversibility means information is structurally destroyed at every forward step. The inverse problem does not merely require computation—it requires recovering information that no longer exists in the current state.

3. The Forward Problem

Theorem 1 (Forward efficiency). *Given an initial configuration C_0 on an n -cell grid, computing $C_t = f^t(C_0)$ requires $O(t \cdot n)$ time and $O(n)$ space.*

Proof. Each application of f examines the 8-neighbourhood of each cell and applies the transition rules. This is $O(n)$ per step. t steps require $O(t \cdot n)$ total work. Two grids of size $O(n)$ suffice (current and next generation), giving $O(n)$ space. $\square$

For a 1000×1000 grid ($n = 10^6$ cells) and $t = 10^6$ steps, this is 10^{12} cell-updates—demanding but finite. A modern GPU performing 10^{10} cell-updates per second completes this in roughly 100 seconds.

The forward problem is easy. This is the fundamental asymmetry.

4. The Inverse Problem

4.1. The $t = 1$ Case

Definition 2 (Predecessor existence problem). *Given a configuration C_1 , does there exist a configuration C_0 such that $f(C_0) = C_1$?*

Theorem 2 (NP-completeness of the predecessor problem [5]). *The predecessor existence problem for CGOL is NP-complete.*

The proof proceeds by reduction from satisfiability (SAT). For each cell c in C_1 , its state is determined by the nine cells in its Moore neighbourhood in C_0 . The CGOL transition rules define a boolean function over those nine inputs. Finding a consistent global assignment of C_0 that satisfies all constraints simultaneously is equivalent to solving a boolean constraint satisfaction problem (CSP), which is NP-complete in general.

This is NP-complete even for $t = 1$. Finding the immediate predecessor of a configuration is already computationally hard.

4.2. Encoding as SAT

For any finite t , the inverse problem can be encoded as a SAT instance. Let $x_{i,j,s}$ be a boolean variable representing the state of cell (i, j) at generation s , for $0 \leq s \leq t$. The

constraints are:

1. **Transition constraints:** for each cell (i, j) and each generation s from 0 to $t - 1$, the CGOL rules must be satisfied. Each such constraint involves 9 variables (the cell and its 8 neighbours at generation s) and produces a constant number of clauses in CNF form.
2. **Target constraints:** the values of $x_{i,j,t}$ must match the target configuration C_t .

The resulting SAT instance has:

$$\text{Variables} = n \cdot (t + 1)$$

$$\text{Clauses} = O(n \cdot t)$$

5. Hardness by Scale

5.1. $t = 10^2$

Grid	SAT Variables	SAT Clauses
100×100	$\sim 10^6$	$\sim 10^7$
1000×1000	$\sim 10^8$	$\sim 10^9$

100 × 100 grid. State-of-the-art SAT solvers (CaDiCaL, Kissat) routinely handle instances of 10^6 variables and 10^7 clauses. The $t = 100$ problem on a small grid is *in principle solvable* with current technology. Practical difficulty varies by target: configurations with many predecessors are easy; those near Garden of Eden territory are hard or unsolvable.

1000 × 1000 grid. At 10^8 variables and 10^9 clauses, we approach the memory limits of SAT solvers on modern hardware ($\sim$ terabyte RAM required). Feasible only for specialised hardware and favourable target configurations.

Verdict: Hard but feasible for small grids; borderline for large grids.

5.2. $t = 10^3$

Grid	SAT Variables	SAT Clauses
100×100	$\sim 10^7$	$\sim 10^8$
1000×1000	$\sim 10^9$	$\sim 10^{10}$

100 × 100 grid. The constraint graph is highly structured but enormously connected. Backtracking SAT solvers and random-walk heuristics lose traction. A satisfying assignment, if one exists, may require exhausting a significant fraction of the 2^{10^4} configuration space.

1000 × 1000 grid. The SAT instance at 10^9 variables cannot be held in memory on existing machines without aggressive compression, which itself imposes computational overhead that renders solving infeasible in practice.

Verdict: Practically intractable for most configurations and grid sizes.

5.3. $t = 10^6$

At one million steps, the inverse problem enters a qualitatively different regime.

Grid	SAT Variables	SAT Clauses
100×100	$\sim 10^{10}$	$\sim 10^{11}$
1000×1000	$\sim 10^{12}$	$\sim 10^{13}$

100×100 grid. 10^{10} boolean variables require at minimum 10^{10} bits = 1.25 terabytes for variable storage alone, before clause storage. The full clause set requires petabytes. This is not a problem of insufficient computing power—it is a problem of representation. The SAT instance cannot be materialised on any existing machine.

1000×1000 grid. The clause count $\sim 10^{13}$ exceeds the estimated number of synapses in the human brain. No physical system on Earth can store or process this instance.

Observation 1. *For $t = 10^6$ on any grid of practical size, the inverse CGOL problem is not merely computationally intractable—it is physically unrepresentable with current technology.*

6. The Information-Theoretic View

6.1. Entropy Loss at Each Step

Each forward step in CGOL is a many-to-one mapping. Information is structurally destroyed. Define the *predecessor count* $P(C)$ of a configuration C as

$$P(C) = |\{C' : f(C') = C\}|.$$

For most configurations, $P(C)$ is large but finite; for Gardens of Eden, $P(C) = 0$.

The t -step inverse image of a target C_t is

$$f^{-t}(C_t) = \{C_0 : f^t(C_0) = C_t\}.$$

This set may be empty (if C_t is unreachable in t steps from any initial configuration), exponentially large, or anything in between. Its cardinality cannot be efficiently computed in general.

6.2. The Attractor Landscape

Under long evolution, CGOL configurations drift toward *attractors*: still lifes, oscillators, and glider streams. Configurations far from these attractors become progressively less reachable as t increases. For a randomly chosen target configuration and large t :

- The target is likely an attractor state or transitional configuration.

- The set $f^{-t}(C_t)$ may be very large (if many initial conditions funnel into this attractor) or empty (if the target is not on any attractor trajectory reachable from typical initial conditions).

For large t , a randomly chosen target configuration is *probably unreachable*. The inverse problem has no solution—not because the computation is hard, but because no answer exists.

7. Connection to Cryptography

The forward-easy, inverse-hard asymmetry of CGOL is structurally identical to the properties required of a *one-way function*—the mathematical object underlying all modern public-key cryptography.

Definition 3 (One-way function (informal)). *A function f is one-way if $f(x)$ is easy to compute for any x , but given $y = f(x)$, finding any x' with $f(x') = y$ is computationally infeasible.*

CGOL forward evolution f^t exhibits this structure. This raises the natural question: can CGOL serve as a cryptographic hash function?

The answer is almost certainly no in practice, for three reasons:

1. **Output size:** CGOL evolves large grids; cryptographic hashing requires compression to a fixed-size output. The compression step must be designed separately and introduces its own security assumptions.
2. **Speed:** CGOL is slow compared to purpose-built hash functions (SHA-256, Blake3). Even GPU-accelerated CGOL cannot approach the throughput of dedicated hash circuits.
3. **Algebraic structure:** The structured local rules of CGOL may allow algebraic or differential attacks not available against pseudorandom functions. No formal hardness reduction from a standard cryptographic assumption exists.

Nevertheless, CGOL is a *natural* one-way process—not designed for cryptography, but exhibiting its essential property as a consequence of irreversibility and Turing completeness. It demonstrates that one-way processes are not exotic constructions but a natural feature of sufficiently complex deterministic systems.

8. The Arrow of Time in Deterministic Systems

CGOL demonstrates that determinism does not imply reversibility. Given the complete current state of the system and perfect knowledge of its laws, we can compute any future state efficiently. We cannot recover the past—not because the laws are probabilistic, but because information has been destroyed at every forward step.

This is not thermodynamics, which is statistical. This is a clean, discrete, finite system exhibiting a rigorous computational arrow of time. The past is not merely unknown—for many configurations, it is computationally unreachable.

Norbert Wiener identified the recovery of information in the presence of interference as the central challenge of cybernetics [6]. The CGOL inverse problem presents a starker version: the interference is not stochastic but structural. Information was not corrupted—it was consumed by the process itself. No amount of computational power recovers a Garden of Eden from its descendants. The information is not hidden. It is gone.

9. Summary

t	Grid	Status	Bottleneck
10^2	100×100	Hard, feasible	SAT solver hours
10^2	1000×1000	Borderline	Memory
10^3	100×100	Practically intractable	Search space
10^3	1000×1000	Practically intractable	Memory + search
10^6	100×100	Unrepresentable	Physical limits
10^6	1000×1000	Far beyond physical	Physical limits

The inverse problem of Conway’s Game of Life transitions through three qualitatively distinct regimes as t grows: NP-hard but tractable with modern SAT solvers; practically intractable due to memory and search space; and physically unrepresentable. The hardness is both computational and informational—entropy is destroyed at every forward step, and many targets have no predecessor at all.

Three rules. You can always go forward. You can almost never go back.

References

- [1] E. Berlekamp, J. Conway, and R. Guy, *Winning Ways for your Mathematical Plays*, Vol. 2. Academic Press, 1982.
- [2] E. R. Banks et al., “Patterns in the Game of Life,” MIT Project MAC Technical Report, 1971.
- [3] E. F. Moore, “Machine models of self-reproduction,” *Proceedings of Symposia in Applied Mathematics*, vol. 14, 1962.
- [4] P. Rendell, “A Universal Turing Machine in Conway’s Game of Life,” *2011 International Conference on High Performance Computing & Simulation (HPCS)*, 2011.
- [5] K. Sutner, “De Bruijn Graphs and Linear Cellular Automata,” *Complex Systems*, vol. 5, 1988.

- [6] N. Wiener, *Cybernetics: or Control and Communication in the Animal and the Machine*, 2nd ed. MIT Press, 1961.